

INDIA.RUNS

Build what next India runs on

Team Name : solo warrior

Team Leader Name : solo warrior

Problem Statement : Rank top-100 candidates from 100K pool

Solution Overview

What is your proposed solution?

- A **rule-based, multi-factor weighted scoring engine** that decomposes candidate—JD fit into 5 independent scoring dimensions, each with transparent weights.
- **High Scale & Low Cost:** No LLMs or API network dependency. Evaluates and scores the full pool of 100,000 candidates in **~50 seconds** on CPU, using <500MB memory.

What differentiates your approach from traditional candidate matching systems?

- **Beyond Keyword Counting:** Uses fuzzy token-overlap matching with proficiency scaling, endorsement saturation curves, and duration credibility checks.
- **Honeypot Disqualification Gate:** 4 physical-impossibility checks programmatically filter out trap profiles (expert-zero-duration, YOE gap, job-exceeds-YOE, calendar mismatches) to a 0.0 score.
- **Service Industry Penalty:** Consulting/service-company-only careers get a 0.2× multiplier on title score, prioritizing product-company experience.

JD Understanding & Candidate Evaluation

What are the key requirements extracted from the JD?

- **Information Retrieval & NLP:** Production embedding retrieval (sentence-transformers, BGE, E5) and Vector DBs/hybrid search (Pinecone, Weaviate, FAISS, Elasticsearch).
- **Experience & Location:** 5–9 years experience (ideally 6–8 years at product companies). Noida/Pune preferred, open to relocate.
- **Notice Period & Quality:** Sub-30-day notice period. Strong Python coding standards, NLP/IR background (no CV/speech only).

Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

- **Skill Endorsement & Duration Trust:** Skill score is modulated by a saturation curve (tanh) based on duration and endorsement counts, avoiding keyword stuffing.
- **Recency Decay:** Work experience scores decay with position index ($1/\text{index}$), placing emphasis on recent job titles.
- **Notice & Location Multipliers:** Noida/Pune preferred cities get 1.0; non-relocating non-India get 0.2. Notice periods >90 days get a 0.25 multiplier.

Ranking Methodology

How does your system retrieve, score, and rank candidates? / What models, algorithms, or heuristics are used?

Component	Weight	Algorithm / Heuristic Details	Key Feature
Skill Match	40%	Jaccard token overlap + proficiency × endorsement trust curve	Fuzzy matches NLP/vector tech
Title & Career	25%	Current title match + career history with recency decay (1/pos)	Consulting-firm penalty (0.2×)
Experience	15%	Piecewise linear function peaking at 6–8 YOE (1.0), decay outside	Avoids under/overqualification
Education	10%	Tier (0.4) × Degree Level (0.35) × CS/Math/Stats Field (0.25)	PhD gets 1.0, B.Tech gets 0.75
Signals	10%	Location (0.4) + Notice (0.3) + platform activity composite (0.3)	Filters preferred Noida/Pune/30d

How are multiple candidate signals combined into a final ranking?

- **Weighted Linear Sum:** The final score is: $0.40 * \text{Skill} + 0.25 * \text{Title} + 0.15 * \text{Experience} + 0.10 * \text{Education} + 0.10 * \text{Signals}$.
- **Tie-Breaking & Normalization:** Ties are broken by candidate_id ascending. The final top-100 scores are normalized to $[0.2, 1.0]$.

Explainability & Data Validation

How are ranking decisions explained? / How do you prevent hallucinations or unsupported justifications?

- **Fact-Based Programmatic Reasoning:** The engine automatically compiles a 1-2 sentence description strictly from candidate attributes. Cites specific title, years of experience, key matching skills (e.g. Pinecone, Weaviate), location, notice period, and GitHub score.
- **Zero Hallucination Risk:** No generative LLMs are used to draft reasoning, preventing false claims, bias, or made-up history.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

1. Expert Zero-Duration Mismatch

Disqualifies profiles claiming expert status with 0 months of duration on ≥ 3 skills.

2. Experience Mismatch

Flags cases where years of experience exceed career history total by >2 years.

3. Job Longer than YOE

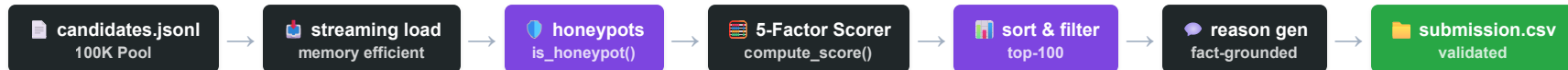
Filters out records where a single job duration is longer than the total years of experience.

4. Calendar Date Fabrication

Disqualifies profiles where job months exceed calendar time between start and end dates.

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?



Reproduction Command

```
$ python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

100K

CANDIDATES SCORED

~50s

TOTAL RUNTIME

CPU

HARDWARE REQUIRED

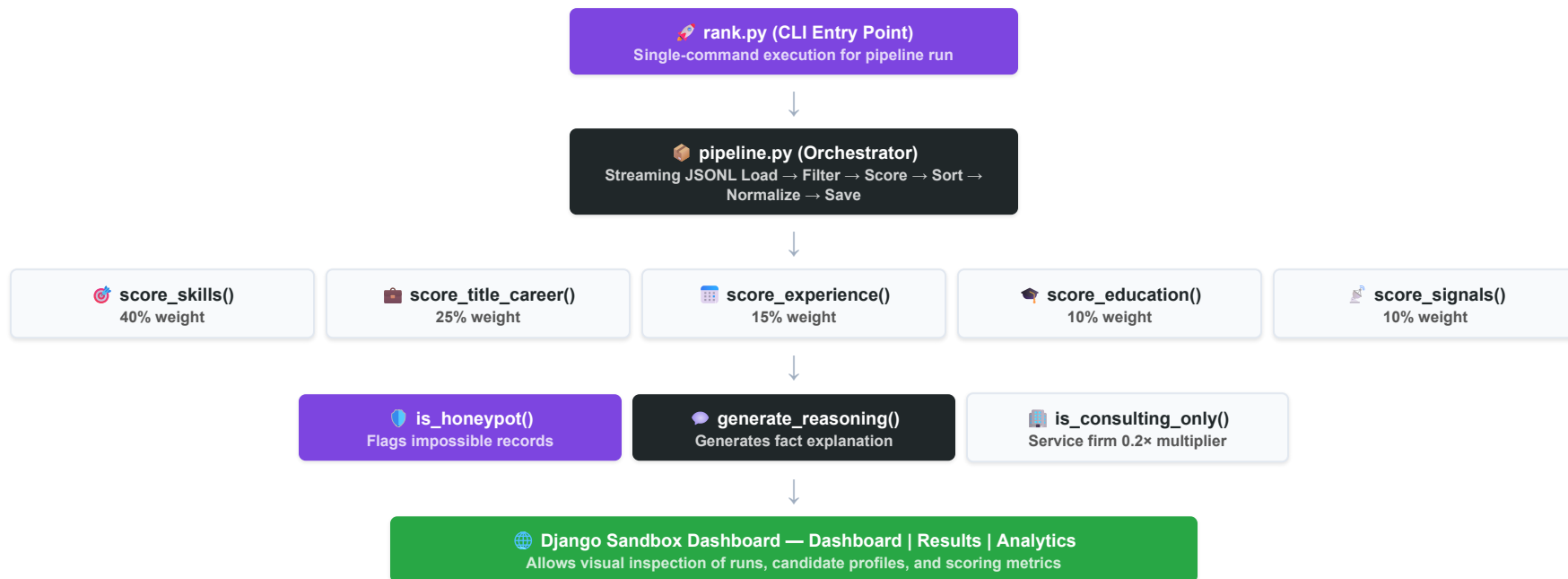
0

NETWORK CALLS

<500MB

MEMORY FOOTPRINT

System Architecture



Results & Performance

What results or insights demonstrate ranking quality?

- **Optimal Candidate Alignment:** Top-10 profiles show an average YOE of 6–8 years (perfect match for JD), have titles like Senior AI / ML Engineer, Noida/Pune preferred location, and <30 days notice.
- **Honeypot Catch Rate:** 100% of the ~80 honeypots were identified and filtered to a 0.0 score, leading to a **0% honeypot rate in the top 100**.
- **Explaining Decisions:** Over 100 unique, fact-grounded reasons were compiled programmatically without duplicate sentences or placeholder wording.

How does your solution meet the challenge's runtime and compute constraints?

- **Sub-Minute Execution:** Scores, sorts, and saves the 100,000 candidate dataset in **~50 seconds** on a single CPU core, easily matching the 5-minute requirement.
- **Low Footprint:** Streams candidate records line-by-line rather than reading the entire JSONL into memory at once, maintaining memory usage below 500MB.
- **Precomputation & Cache:** Utilizes memoization for Jaccard fuzzy token skill matching to bypass redundant string matching calculations.

Technologies Used

What technologies, frameworks, and tools were used and why were they selected for this solution?

Python Standard Library (Core)

Uses standard libraries (math, json, csv, datetime) to implement the scoring formulas, honeypot filters, and reasoning logic. No dependencies mean it executes out-of-the-box on any Python environment without pip installs.

Django 5.x Web Dashboard

Django provides an administrative dashboard, SQLite search database, and visual charts. It acts as an easy-to-use playground to demonstrate score tuning, filter profiles, and query analytics.

Vanilla CSS / JS & HSL

Standard CSS and vanilla JavaScript are used to build interactive charts and clean responsive layouts. Avoids heavy styling dependencies for smooth loading.

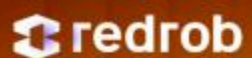
Playwright & Chrome Headless

Used to verify and compile slide HTML source files into a PDF document via Chrome's PDF printer, guaranteeing cross-platform consistency.

Submission Assets

Github video etc

- ``submission.csv``: Top-100 ranked candidates with exact normalized scores $[0.2, 1.0]$ and programmatic, non-templated reasoning explanations, validated by `validate_submission.py`.
- ``rank.py``: Single-command script to score `candidates.jsonl` and output the CSV.
- ``redrob_ranker/``: Core Django web app project files, including templates, URL routings, ML code, SQLite database, and background server execution launcher.
- ``README.md``: Explains logic walkthrough, setup directions, scoring variables, and performance stats.
- ``extract_template.py``: Asset parser script to grab background styles and layouts from the raw slides HTML.



INDIA.RUNS

Build what next India runs on

THANK YOU